Pedagogical Defeasible Reasoning: Literature Review Scaffold - Final

Nikita Martin
mrtnik003@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Defeasible reasoning is a form of nonmonotonic reasoning that allows conclusions to be revised when new or more specific information arises, closely mirroring how humans reason in everyday life. Despite its growing importance in AI and knowledge representation, defeasible reasoning has not yet been widely incorporated into logic education. In this literature review, we examine the theoretical and practical foundations relevant to the development of a pedagogical tool for defeasible reasoning. We begin by introducing classical propositional logic and identifying the limitations of monotonic reasoning, motivating the need for defeasible reasoning. We examine the KLM framework as the formal foundation for defeasible reasoning, before presenting the Rational Closure algorithm as a method for computing defeasible entailment, which is grounded in the observation that through materialisation, defeasible reasoning can be reduced to a series of classical entailment checks. Extensions to Rational Closure, namely Lexicographic Closure and Relevant Closure, are briefly surveyed. We also evaluate existing pedagogical tools for classical logic and defeasible reasoners, revealing that to the best of our knowledge, no dedicated pedagogical tool for defeasible reasoning currently exists.

Keywords

Propositional Logic, Defeasible Reasoning, KLM Framework, Rational Closure, Pedagogical Tools, Logic Education

1 Introduction

Defeasible reasoning is a form of non-monotonic reasoning in which conclusions are drawn tentatively and may be revised when new or more specific information is provided [14, 20]. Unlike classical monotonic reasoning, where conclusions remain valid once derived, defeasible reasoning allows conclusions to be retracted as additional information is introduced, more closely mirroring how humans reason in everyday life [14].

Despite its growing importance in AI, defeasible reasoning has not yet been widely incorporated into logic education. To the best of our knowledge, no dedicated pedagogical tool for defeasible reasoning exists. Existing tools, such as Iltis focus on classical monotonic reasoning, while tools such as the Tweety Project are designed for research use instead of educational purposes, creating a gap between tools used for teaching logical reasoning and systems capable of performing defeasible reasoning.

This literature review examines the foundations of defeasible reasoning, analyses the Rational Closure algorithm as a method of computing defeasible reasoning, and evaluates existing tools to identify the gap that motivates the development of educational tools for defeasible reasoning. This review is structured as follows: Section 2 introduces classical reasoning and its limitations, Section 3 covers the foundations of defeasible reasoning, including the KLM framework. Section 4 examines the Rational Closure algorithm, and Section 5 examines existing tools and identifies the gap. Section 6 concludes by summarising the key findings of the review and identifying the gap that motivates the further work.

2 Foundations of Classical Reasoning

2.1 Propositional Logic

Propositional logic is a formal system used to reason about knowledge and information by abstracting natural language into symbolic representations. It is defined by propositional atoms, which represent basic statements, and a set of connectives, which allow for the construction of complex formulas where the truth of the composite statements depends on the truth of their base components and the semantics of the connective used [14].

The **syntax** of propositional logic consists of propositional atoms, which are the simplest statements which are often represented as variables, such as p, q, r, or shorthand words like $bird$ [10, 14]. They represent propositions which are either true, denoted T, or false, denoted as F. The set of all propositions is denoted as P, and the atoms are combined using connectives to construct a set of *well-formed formulas*, referred to as propositional formulas, or just formulas. The following are the connectives used in the literature:

- $\neg$ (negation)
- $\wedge$ (and)
- $\vee$ (or)
- $\rightarrow$ (if then)
- $\leftrightarrow$ (if and only if)

The set of all such formulas is denoted as $\mathcal{L}$, and the elements of $\mathcal{L}$ will be denoted by lowercase Greek letters such as α, β, γ. Formally, for any $p \in P$ and $\alpha, \beta \in \mathcal{L}$, the formulas $p, \neg\alpha, \alpha \wedge \beta, \alpha \vee \beta, \alpha \rightarrow \beta$, and $\alpha \leftrightarrow \beta$ are all elements of $\mathcal{L}$. The constants $\top \in \mathcal{L}$ and $\bot \in \mathcal{L}$ are also elements of $\mathcal{L}$, where $\top$ represents a tautology, which is always true, and $\bot$ represents a contradiction, which is always false [2, 10, 14].

The **semantics** defines the meaning of truth within the logic through **valuations**, also referred to as a world or interpretation. A **valuation** is a function $u : P \rightarrow \{T, F\}$ that assigns a truth value to every propositional atom in the language, and the set of valuations is denoted as U [10, 14]. For example, a valuation u could assign $u(bird) = T$ and $u(flies) = F$, representing a world in which something is a bird but does not fly. If a valuation makes a formula $\alpha \in \mathcal{L}$ true, that is, α evaluates to true, it is said to *satisfy* that formula, denoted $u \models \alpha$. **Satisfaction** is defined recursively over the connectives, for example, $u \models \alpha \rightarrow \beta$ if and only if $u \not\models \alpha$ or

$u \models \beta$ [10, 14]. Any valuation satisfying α is called a model of α, and the set of all models is denoted $\hat{\alpha}$.

From satisfaction, **classical entailment**, can be defined as $\alpha \models \beta$, which holds if and only if every model of α is also a model of β, such that $\hat{\alpha} \subseteq \hat{\beta}$. $\models$ is a meta-level notation, it is a statement about the language $\mathcal{L}$ and not a connective inside of it. This contrasts with $\rightarrow$, which is an object-level notation, whose meaning is fixed inside of the truth table [14]. The distinction of the two becomes significant when defeasible implication is introduced in Section 3. A formula α is said to be satisfiable if there exists at least one valuation u such that $u \models \alpha$, and unsatisfiable if no such valuation exists. A useful property of classical entailment is that $\alpha \models \beta$ if and only if $\alpha \wedge \neg\beta$ is unsatisfiable [10, 14]. This means that entailment checking can be reduced to satisfiability checking, which is a point that becomes relevant when examining the Rational Closure algorithm in Section 3.

2.2 Knowledge Bases

In propositional logic, a knowledge base, usually denoted as K, is a finite set of formulas, which is intended to represent a collection of facts about a world or a specific domain [14]. Instead of looking at a single statement, knowledge bases allow us to group multiple pieces of information together about what they collectively imply. For example, a simple knowledge base about birds could contain the formulas:

$$K = \{bird \rightarrow flies, bird \rightarrow wings, penguin \rightarrow bird\}$$

A valuation u is said to satisfy a knowledge base K if it satisfies every formula within K, and K is said to be satisfiable if at least one such valuation exists. Any valuation that satisfies the entire knowledge base is called a **model** of K. Each formula in the knowledge base can be thought of as a *filter*, such that every time a fact is added to K, the set of possible worlds becomes restricted, only to those where that fact is true. This concept of satisfaction allows for the definition of classical entailment, where $K \models \alpha$ holds if and only if $K \cup \{\neg\alpha\}$ is unsatisfiable, meaning that every model of K is also a model of α. However, this notion of classical entailment satisfies a property called monotonicity, which reveals a limitation when reasoning about exceptions [14].

2.3 Monotonic Reasoning and Its Limitations

Logical reasoning frameworks are used in AI to represent knowledge and allow inferences to be drawn. Classical reasoning is the dominant form of logic, that is built on Tarskian notions of consequence, ensuring that each inference has maximal support from the explicit knowledge provided. Classical reasoning is monotonic whereby conclusions follow from a set of premises, which when derived, remains valid, even when new premises or information is provided [8, 14]. In a monotonic system, the set of inferences can only expand when new knowledge is added, and it is impossible for the new information to cause the retraction of a previously held conclusion. At first glance, this seems reasonable, for example, if it is known that all mammals breathe air, and that whales are mammals, then it follows that whales breathe air. Learning that whales live in the ocean should not change this conclusion. [10, 14]. However, monotonicity presents significant limitations when it comes to dealing with real-world information and "common sense" reasoning, that is, when the knowledge base contains exceptions. Consider the knowledge base:

$$K = \{bird \rightarrow flies, penguin \rightarrow bird, penguin \rightarrow \neg flies\}$$

Using classical reasoning, from $bird \rightarrow flies$ and $penguin \rightarrow bird$, it is possible to derive $penguin \rightarrow flies$. However, K also contains $penguin \rightarrow \neg flies$, directly contradicting this conclusion. This directly illustrates the limitation of classical reasoning, where it has no mechanism for handling "exceptional" information, that is, cases where the derived information violates the general rule. Instead of treating the penguin as an exception to the general rule that birds fly, classical reasoning attempts to hold both conclusions simultaneously, resulting in an inconsistent knowledge base.

In everyday reasoning, humans naturally revise their conclusions upon learning new information, upon learning something is a penguin, one would naturally withdraw the conclusion that penguins fly. The inflexible and monotonic nature of classical reasoning does not allow conclusions such as $penguin \rightarrow flies$ to be retracted once new information is presented, and the only resolution available is inconsistency [8, 10, 14]. To avoid contradictions in a monotonic system, exceptions must be handled explicitly within the formulas themselves (*e.g.,* "birds that are not penguins, ostriches, or kiwis fly"). However, as the number of exceptions in the knowledge base grows, the knowledge base becomes larger and more difficult to maintain.

These limitations motivate the need for a form of reasoning that allows for conclusions to be drawn tentatively and revised when more specific or contradictory information is encountered. Such frameworks fall under the category of non-monotonic reasoning, of which defeasible reasoning is an example [14].

3 Foundations of Defeasible Reasoning

The limitations of classical monotonic reasoning identified in the previous section, motivate the need for a more flexible form of reasoning that can handle exceptions and incomplete information [14]. This section introduces defeasible reasoning as a solution to these limitations. First, non-monotonic reasoning is introduced as the broader category under which defeasible reasoning falls, and then defeasible reasoning is defined more precisely, followed by an introduction to the KLM [14] framework which provides the formal foundations for defeasible reasoning.

3.1 Nonmonotonic Reasoning

Nonmonotonic reasoning is a branch of logic that formalizes "common sense" reasoning, by allowing the inferences to be tentative rather than permanent [21]. In classical propositional logic, knowledge is strictly monotonic, meaning that once a conclusion is derived, it remains valid regardless of what new information is added [8]. However, non-monotonic systems allow new conclusions to be inferred, which may override or retract conclusions previously derived, when new or contradictory information is encountered, closely mirroring how humans reason in everyday life [14, 20].
Central to nonmonotonic reasoning is the notion of typicality, which is the idea that instead of expressing absolute rules or specific information, a reasoning system should be able to explain what is typically or normally the case [16, 21]. Classical logic uses

universal statements such as "all birds fly", which fails in the presence of exceptions such as penguins or ostriches. Typicality allows for "common sense" generalizations such as "birds typically fly", which holds in a more general case and remains useful even in the presence of exceptions such as penguins, but may be overridden or retracted when more specific information is presented [14, 21]. This shift from absolute truths to typical properties allows systems to handle exceptions gracefully, retracting specific conclusions about an individual like "Tweety flies", upon learning that it is an exception like "Tweety is a penguin", without having to discard the general rule itself. This notion of typicality is central to the KLM framework.

Non-monotonic reasoning emerged as a field of research in the 1980s, motivated by the limitations of classical reasoning in representing common sense knowledge [14]. A number of frameworks were developed to formalize different patterns of nonmonotonic reasoning, including Default Logic [16, 21], Circumscription [14], and the preferential approach of Kraus, Lehmann and Magidor [14, 16].

3.2 Defeasible Reasoning

Among these patterns of nonmonotonic reasoning, defeasible reasoning has emerged as a particularly prominent approach, characterised by its ability to draw conclusions tentatively and to revise them in light of new or more specific information. [14, 20]. Defeasible reasoning is a form of nonmonotonic reasoning whereby when conclusions are drawn and accepted, they may be changed if new information or evidence is presented. Kaliski [14] describes defeasible reasoning as the pattern of making assumptions based on given knowledge, and then revising them upon learning new information. An argument is considered defeasible when it provides rational grounds for accepting a conclusion, but that conclusion remains retractable when new information is presented, lacking deductive validity.

What distinguishes defeasible reasoning from nonmonotonic reasoning more broadly is the notion of *defeat*, whereby conclusions are *overridden* by more specific or stronger information that takes priority and not just simply removed, while the general rule that produced them remains valid [14, 20]. Pollock [20] identifies two distinct ways in which an inference can be defeated: **rebutting defeaters**, which directly contradict a conclusion, supporting its negation, and **undercutting defeaters**, which attacks the reasoning that led to it, capturing the different ways in which conclusions may be revised in defeasible reasoning [15, 20]. This notion of defeat is captured through *defeasible conditionals*.

Defeasible reasoning uses a distinct notation to classical propositional logic to distinguish defeasible conditionals from classical ones [14, 16]:

- $\alpha \rightarrow \beta$ - classical implication, meaning that α *always* implies β.
- $\alpha \mid\sim \beta$ - defeasible implication, meaning that α *typically* implies β.

Returning to the bird example, the classical statement $bird \rightarrow flies$ introduces inconsistency when exceptions such as penguins are introduced, whereas the defeasible statement $bird \mid\sim flies$ remains valid as it holds in the typical case and can be overridden by more specific information.

3.3 KLM Framework

The KLM framework, which is proposed by Kraus, Lehmann, and Magidor [16], formalises the notion of typicality, by extending propositional logic with defeasible conditionals of the form $\alpha \mid\sim \beta$ where $\alpha, \beta \in \mathcal{L}$, read as *"typically, if α occurs, then β occurs"* [18]. To give semantics to these conditionals, this framework introduces a preferential consequence relation over propositional logic, defining how to reason in a nonmonotonic way over the propositional language. This framework forms the foundation of defeasible reasoning, as it allows both general rules and exceptions to be stored in a defeasible knowledge base K without resulting in a contradiction[14, 16]. The semantics of these conditionals are given by ranked interpretations which provide a formal structure for reasoning about typicality and exceptionality.
A ranked interpretation $\mathcal{R}$ is a formal semantic structure used in the KLM framework to give meaning to defeasible conditionals [5, 18]. Valuations with a lower rank are considered more typical or normal, while those with a higher rank are considered more exceptional or atypical, and valuations assigned rank ∞ are considered impossible worlds [5]. A ranked interpretation $\mathcal{R}$ satisfies a defeasible conditional $\alpha \mid\sim \beta$ if all the most typical valuations satisfying α also satisfy β, capturing the intuition that β typically follows from α in the normal case [5, 16, 18]. These ranked interpretations provide the semantic foundation for Rational Closure, as rankings over valuations directly correspond to the base ranks that the algorithm computes over defeasible statements.

Defeasible entailment, denoted as $\mid\approx$, is defined over ranked interpretations. The expression of the form $\mathcal{K} \mid\approx \alpha \mid\sim \beta$ means that, according to the knowledge base $\mathcal{K}$, "β typically follows from α" [14]. Unlike classical monotonic entailment, defeasible entailment is nonmonotonic, allowing conclusions to be overridden or retracted if they are defeated by the discovery of exceptions or more specific information [14, 15]. This makes defeasible entailment a more flexible form of reasoning, as it is capable of handling exceptions and incomplete information that classical reasoning cannot handle [14].

An entailment relation is considered LM-rational if it satisfies the KLM properties proposed by Kraus, Lehmann and Magidor [16], and later extended by Lehmann and Magidor [18]. However, ranked entailment does not satisfy rational monotonicity and remains monotonic, making it insufficient as a truly nonmonotonic entailment relation [4, 5, 14]. Despite this limitation, it serves as the monotonic core upon which the appropriate forms of defeasible entailment are built [4, 5]. This motivates the need for stronger forms of defeasible entailment such as the Rational Closure, which is examined in Section 4.

4 Computing Defeasible Reasoning

The theoretical foundations of defeasible reasoning established in the previous section raise a question of how defeasible reasoning can be computed. Since the intended audience has a background in propositional logic, it is useful to bridge their existing knowledge of classical reasoning with defeasible reasoning. While ranked interpretations provide the semantic grounding for Rational Closure, the

algorithm itself is computed syntactically through *materialisation*, also referred to as the *material counterpart*, which is the process of converting a defeasible statement into a classical propositional formula, allowing defeasible reasoning to be grounded in classical entailment checking [12, 14]. Specifically, the materialisation of a defeasible implication $\alpha \mathrel{|\!\sim} \beta$, is the classical implication, $\alpha \rightarrow \beta$, and the materialisation of a defeasible knowledge base $\mathcal{K}$, denoted $\overrightarrow{\mathcal{K}}$, is the set of all such counterparts [14, 18].

This conversion can be made through the use of defeasible reasoning algorithms, which provides a way for defeasible knowledge bases to be converted into a form that can be reasoned using classical logic. One such algorithm is the Rational Closure Algorithm.

4.1 Rational Closure

A defeasible knowledge base is a finite set of defeasible statements, which may also contain classical statements [14, 16]. Rational Closure, first defined by Lehmann and Magidor [18], is a form of defeasible entailment which serves as a foundation for computing defeasible reasoning [4, 14]. It is the most conservative form of LM-rational defeasible entailment relation, as it draws the minimum number of defeasible conclusions that are considered reasonable, and forms the nonmonotonic foundation upon which bolder entailment relations are built [4]. The central idea of Rational Closure is to rank defeasible statements based on how exceptional they are, where general statements are assigned to lower ranks, more specific exceptions are assigned to higher ranks, and the classical statements are assigned to rank ∞ [14, 18].

Rational Closure can be characterised both semantically, through the notion of a minimal ranked model, and syntactically, through the notion of base ranks, and these two characteristics are provably equivalent [4, 14]. At a semantic level, Rational Closure is defined in terms of the minimal ranked model of $\mathcal{K}$, which is the unique ranked interpretation that makes every valuation as typical as $\mathcal{K}$ allows [4, 14]. However, since Rational Closure can be characterised syntactically through the notion of base ranks, it can be computed algorithmically by reducing defeasible entailment checking to a series of classical entailment checks on the materialisation $\overrightarrow{\mathcal{K}}$ [4, 14, 18]. This syntactic characterisation is what makes the materialisation introduced earlier central to the algorithm. The remainder of this section focuses on this syntactic characterisation and the algorithms that realise it.

The first step in determining how exceptional a formula is relative to a knowledge base $\mathcal{K}$. A propositional formula α is said to be exceptional with respect to $\mathcal{K}$ if it is false in all the most typical worlds in every ranked model of $\mathcal{K}$ [4, 14]. Formally, α is exceptional with respect to $\mathcal{K}$ if $\mathcal{K} \mathrel{|\!\approx_R} \top \mathrel{|\!\sim} \neg\alpha$. Using the materialisation, $\overrightarrow{K}$, this can be reduced to a classical entailment check where α is exceptional with respect to $\mathcal{K}$ if and only if $\overrightarrow{\mathcal{K}} \models \neg\alpha$ [4, 18]. In simpler terms, α is exceptional if the materialised knowledge base classically entails its negation. Since classical entailment can be checked using a SAT solver, exceptionality reduces to a satisfiability check, requiring no defeasible reasoning machinery at all.

To illustrate, consider $\mathcal{K} = \{b \mathrel{|\!\sim} f,\ p \rightarrow b,\ p \mathrel{|\!\sim} \neg f\}$, with materialisation: $\overrightarrow{\mathcal{K}} = \{b \rightarrow f,\ p \rightarrow b,\ p \rightarrow \neg f\}$. To check whether p is exceptional, we ask whether $\overrightarrow{\mathcal{K}} \models \neg p$. From $\overrightarrow{\mathcal{K}}$, we can derive $p \rightarrow f$ from $p \rightarrow b$ and $b \rightarrow f$. However, $\overrightarrow{\mathcal{K}}$ also contains $p \rightarrow \neg f$, meaning that assuming that p is true leads to both f and $\neg f$ simultaneously, which is a contradiction. Therefore $\overrightarrow{\mathcal{K}} \models \neg p$, and p is exceptional. Similarly, by checking whether b is exceptional, we ask whether $\overrightarrow{\mathcal{K}} \models \neg b$. Since there is no contradiction arising when assuming b is true, $\overrightarrow{\mathcal{K}} \not\models \neg b$, and therefore b is not exceptional.

Using exceptionality, we iteratively compute a sequence $\mathcal{E}_0^{\mathcal{K}}, \ldots, \mathcal{E}_\infty^{\mathcal{K}}$ where $\mathcal{E}_0^{\mathcal{K}} = \mathcal{K}$ and each subsequent level only keeps defeasible statements with exceptional antecedents, terminating when no further changes occur [4, 14].

At each step, the function $\varepsilon(\mathcal{K})$ extracts only the defeasible statements whose antecedents are exceptional with respect to the current knowledge base, removing those that are no longer exceptional, only keeping the most exceptional defaults. This process is guaranteed to terminate as $\mathcal{K}$ is finite [4]. The base rank of a formula α, denoted $br_{\mathcal{K}}(\alpha)$, is the smallest r such that α is no longer exceptional with respect to $\mathcal{E}_r^{\mathcal{K}}$ [4, 14]:

$$br_{\mathcal{K}}(\alpha) \equiv_{def} \min\{r \mid \mathcal{E}_r^{\mathcal{K}} \not\mathrel{|\!\approx_R} \top \mathrel{|\!\sim} \neg\alpha\}$$

The base rank captures how exceptional a formula is, where the lower the rank, the more general and defeasible the statement, while a higher rank indicates a more specific exception. Classical statements are assigned rank ∞ as they remain valid in every world and are never exceptional. The base rank of a defeasible implication is defined as the base rank of its antecedent: $br_{\mathcal{K}}(\alpha \mathrel{|\!\sim} \beta) \equiv_{def} br_{\mathcal{K}}(\alpha)$ [4].

Returning to our example, $\mathcal{K} = \{b \mathrel{|\!\sim} f,\ p \rightarrow b,\ p \mathrel{|\!\sim} \neg f\}$, we compute the sequence as follows. Starting with $\mathcal{E}_0^{\mathcal{K}} = \mathcal{K}$, we already concluded that p is exceptional and b is not. Applying ε keeps only statements with exceptional antecedents, giving $\mathcal{E}_1^{\mathcal{K}} = \{p \mathrel{|\!\sim} \neg f\}$. The classical statement $p \rightarrow b$ is set aside and will be assigned to R_∞ at the end. At the next step, the materialisation of $\mathcal{E}_1^{\mathcal{K}}$ is $\{p \rightarrow \neg f\}$. Checking whether p is still exceptional, does $\{p \rightarrow \neg f\} \models \neg p$? It does not, so there is no contradiction assuming p is true in isolation, so it is no longer exceptional and $\mathcal{E}_2^{\mathcal{K}} = \emptyset$. Since $\mathcal{E}_2^{\mathcal{K}} = \emptyset$, no further exceptional statements remain, so the sequence terminates with $n = 2$.

The base ranks are therefore:

- $br_{\mathcal{K}}(b \mathrel{|\!\sim} f) = br_{\mathcal{K}}(b) = 0$: b was not exceptional at level 0, so $b \mathrel{|\!\sim} f$ is assigned base rank 0
- $br_{\mathcal{K}}(p \mathrel{|\!\sim} \neg f) = br_{\mathcal{K}}(p) = 1$: p was no longer exceptional at level 1, so $p \mathrel{|\!\sim} \neg f$ is assigned base rank 1
- $br_{\mathcal{K}}(p \rightarrow b) = \infty$: classical statement, always holds

These base ranks partition the materialisation $\overrightarrow{\mathcal{K}}$ into ranked sets as follows:

Rank	Materialised Statements
∞	$p \rightarrow b$
1	$p \rightarrow \neg f$
0	$b \rightarrow f$

Table 1: Base rank partition of $\overrightarrow{\mathcal{K}}$

This partition is exactly the output that the BaseRank algorithm produces, which is described below.
The Base Rank algorithm formalises the process of computing base ranks. It takes a defeasible knowledge base $\mathcal{K}$ as input and produces an ordered tuple $(R_0, \ldots, R_{n-1}, R_\infty, n)$ as output, where each R_i is the set of materialised statements assigned a base rank i, and R_∞ contains the classical statements [4, 14].

Algorithm 1 BaseRank [4]

Require: A knowledge base $\mathcal{K}$
Ensure: An ordered tuple $(R_0, \ldots, R_{n-1}, R_\infty, n)$
$i := 0$
$E_0 := \overrightarrow{\mathcal{K}}$
repeat
  $E_{i+1} := \{\alpha \rightarrow \beta \in E_i \mid E_i \models \neg\alpha\}$
  $R_i := E_i \setminus E_{i+1}$
  $i := i + 1$
until $E_{i-1} = E_i$
$R_\infty := E_{i-1}$
if $E_{i-1} = \emptyset$ **then**
  $n := i - 1$
else
  $n := i$
end if
return $(R_0, \ldots, R_{n-1}, R_\infty, n)$

This algorithm begins by setting E_0 to the full materialisation $\overrightarrow{K}$. At each iteration, it identifies which statements in E_i have exceptional antecedents, where $\alpha \rightarrow \beta \in E_i$ satisfy $E_i \models \neg\alpha$, and only keeps those in E_{i+1}. The statements removed at each step, $R_i = E_i \setminus E_{i+1}$, are those whose antecedents are no longer exceptional at level i and are therefore assigned base rank i. This continues until no further changes occur, which is the point at which the loop terminates. The statements remaining in the final set when the loop terminates are assigned to R_∞, representing classical statements. The algorithm terminates since $\mathcal{K}$ is finite [4].

Returning to our example $\mathcal{K} = \{b \mid\sim f,\ p \rightarrow b,\ p \mid\sim \neg f\}$, we trace the BaseRank algorithm as follows:

- **1st iteration:** $E_0 = \overrightarrow{\mathcal{K}} = \{b \rightarrow f,\ p \rightarrow b,\ p \rightarrow \neg f\}$. Checking exceptionality: $E_0 \models \neg p$, so $p \rightarrow \neg f$ is exceptional and moves to E_1. $E_0 \not\models \neg b$, so $b \rightarrow f$ is not exceptional and is assigned $R_0 = \{b \rightarrow f\}$. The classical statement $p \rightarrow b$ has antecedent p which is exceptional, so it also moves to E_1.
- **2nd iteration:** $E_1 = \{p \rightarrow b,\ p \rightarrow \neg f\}$. Checking exceptionality: does $E_1 \models \neg p$? No, $\{p \rightarrow b,\ p \rightarrow \neg f\}$ does not entail $\neg p$. So p is no longer exceptional at level 1, so $p \rightarrow \neg f$ is assigned $R_1 = \{p \rightarrow \neg f\}$. The classical statement $p \rightarrow b$ remains and is assigned to $R_\infty = \{p \rightarrow b\}$. Since $E_2 = \emptyset$, no further exceptional statements remain, the loop terminates.

The algorithm returns the tuple $(\{b \rightarrow f\}, \{p \rightarrow \neg f\}, \{p \rightarrow b\}, 2)$, corresponding to the partition shown in Table 1. Note that the algorithm requires only classical entailment checks at each step, confirming that computing base ranks is no harder than classical reasoning [4].

Using base ranks, Rational Closure entailment can be defined syntactically, where a defeasible implication $\alpha \mid\sim \beta$ is in the rational closure of $\mathcal{K}$ if and only if [4]:

$$\mathcal{K} \mid\approx_{RC} \alpha \mid\sim \beta \iff br_{\mathcal{K}}(\alpha) < br_{\mathcal{K}}(\alpha \wedge \neg\beta) \quad \text{or} \quad br_{\mathcal{K}}(\alpha) = \infty$$

This means that β typically follows from α if α is ranked strictly lower than $\alpha \wedge \neg\beta$, meaning that the most typical worlds containing α satisfy β, and the worlds where α holds but β fails are more exceptional. If α has infinite rank, it is a classical statement and is always in the rational closure.

With the base rank partition established by the BaseRank algorithm, the Rational Closure algorithm can now answer defeasible queries. It takes a knowledge base $\mathcal{K}$ and a query $\alpha \mid\sim \beta$ as input, and returns true if and only if the query is in the rational closure of $\mathcal{K}$, and false otherwise [4, 14].

Algorithm 2 RationalClosure [4]

Require: A knowledge base $\mathcal{K}$ and a query $\alpha \mid\sim \beta$
Ensure: **true** if $\mathcal{K} \mid\approx_{RC} \alpha \mid\sim \beta$, and **false** otherwise
$(R_0, \ldots, R_{n-1}, R_\infty, n) :=$ BaseRank($\mathcal{K}$)
$i := 0$
$R := \bigcup_{j=0}^{n-1} R_j$
while $R_\infty \cup R \models \neg\alpha$ **and** $R \neq \emptyset$ **do**
  $R := R \setminus R_i$
  $i := i + 1$
end while
return $R_\infty \cup R \models \alpha \rightarrow \beta$

In simple terms, the algorithm works as follows. It starts with the collection of ranked statements in R, combined with classical statements in R_∞. Then it checks whether α is exceptional with respect to the current set, that is whether $R_\infty \cup R \models \neg\alpha$. If α is exceptional, the algorithm removes the lowest ranked statements R_i from R and checks again. This process repeats, removing ranks from the bottom until α is no longer exceptional or R is empty. At that point, the algorithm checks whether the remaining statements classically entail $\alpha \rightarrow \beta$. If they do, the query is in the rational closure, and the algorithm returns true, and if they don't, it return false [4, 14].
To illustrate this, we apply the algorithm to two queries against our knowledge base $\mathcal{K} = \{b \mid\sim f,\ p \rightarrow b,\ p \mid\sim \neg f\}$, using the partition from Table 1: $R_0 = \{b \rightarrow f\}$, $R_1 = \{p \rightarrow \neg f\}$, $R_\infty = \{p \rightarrow b\}$.

Query 1: Does $\mathcal{K} \mid\approx_{RC} b \mid\sim f$: do birds typically fly? Start with $R = \{b \rightarrow f,\ p \rightarrow \neg f\}$ and $R_\infty = \{p \rightarrow b\}$, and check if $R_\infty \cup R \models \neg b$. There is no contradiction from assuming b is true. So b is not exceptional. Check if $R_\infty \cup R \models b \rightarrow f$. Yes, $b \rightarrow f$ is directly in R. The algorithm returns **true**, concluding that birds typically fly.

Query 2: Does $\mathcal{K} \mid\approx_{RC} p \mid\sim f$: do penguins typically fly? Start with $R = \{b \rightarrow f,\ p \rightarrow \neg f\}$ and $R_\infty = \{p \rightarrow b\}$. Check if $R_\infty \cup R \models \neg p$. Yes, assuming p is true leads to a contradiction. So p is exceptional. Remove $R_0 = \{b \rightarrow f\}$ from R. $R = \{p \rightarrow \neg f\}$. Check if $R_\infty \cup R \models \neg p$? No, $\{p \rightarrow b,\ p \rightarrow \neg f\}$ does not entail $\neg p$. So p is no longer exceptional. Check if $R_\infty \cup R \models p \rightarrow f$? No, $\{p \rightarrow b,\ p \rightarrow \neg f\}$ does not entail $p \rightarrow f$, it entails the opposite

$p \rightarrow \neg f$. The algorithm returns **false**, concluding penguins do not typically fly.

Freund [9] proved that the Rational Closure algorithm is sound and complete, and returns true if and only if $\mathcal{K} \mid\!\approx_{RC} \alpha \mid\sim \beta$ [4, 9]. Since the algorithm makes only a polynomial number of calls to a classical entailment checker in the size of $\mathcal{K}$, computing Rational Closure is no harder than checking classical entailment [4, 6].

While Rational Closure is complete and computationally tractable, it has a limitation known as the *drowning problem* [5, 14]. When a subclass is exceptional with respect to one property, it loses inheritance of all typical properties of its super class, even those unrelated to the conflict. For example, consider extending $\mathcal{K}$ with $b \mid\sim w$, representing that birds typically have wings. Because penguins are exceptional birds with respect to flying, Rational Closure discards the entire rank containing $b \mid\sim f$ and $b \mid\sim w$ when evaluating penguin queries. As a result, it fails to conclude $p \mid\sim w$, that penguins typically have wings, even though nothing in the knowledge base $\mathcal{K}$ contradicts this [14]. This motivates bolder forms of defeasible entailment which will be examined briefly in the following section.

4.2 Other Defeasible Reasoning Algorithms

While Rational Closure forms the nonmonotonic core of defeasible entailment, two broader extensions have been developed to address the drowning problem.

The **Lexicographic Closure**, proposed by Lehmann [17], addresses the drowning problem of the Rational Closure algorithm by removing only the specific statements responsible for conflict instead of discarding the entire rank. It considers worlds that violate fewer defaults as more typical, allowing properties unrelated to the exception to be inherited [14, 17]. Lexicographic Closure is LM-rational and strictly extends Rational Closure where every inference drawn by Rational Closure is also drawn by Lexicographic Closure, but not the other way around [4, 17]. For example, while Rational Closure does not conclude that penguins typically have wings, Lexicographic Closure does, since nothing in the knowledge base contradicts this [4].

Relevant closure, proposed by Casini et al. [3] , takes a different approach by only identifying defeasible statements relevant to a given query, which are those involved in making the query antecedent exceptional and applying a modified version of the Rational Closure algorithm to those statements. Unlike Lexicographic Closure, Relevant Closure does not satisfy all KLM properties and is therefore not LM-rational, though it is inferentially stronger than Rational Closure, but weaker than Lexicographic Closure [3].

Both Lexicographic and Relevant Closure build upon Rational Closure as their foundation, further motivating its central role in defeasible reasoning and the development of the pedagogical tool in this work.

5 Defeasible Reasoning in Education

There is a growing importance of defeasible reasoning in AI [14, 16], with applications in legal reasoning [7], medical reasoning [19], and knowledge representation [14], further motivating the need for accessible tools which allows students and researchers to engage more with defeasible reasoning concepts. However, teaching nonmonotonic reasoning remains challenging, as students often struggle with understanding logic or monotonicity [11] before encountering its defeasible extension. While some defeasible reasoners and pedagogical tools for classical logic exist, to the best of our knowledge, there is no dedicated pedagogical tool designed specifically for defeasible reasoning. These existing tools are primarily for research purposes and lack the infrastructure needed to support student learning.

5.1 Existing Pedagogical Tools for classical logic

A notable example of a pedagogical tool for classical logic is Iltis, which is a web-based interactive tool for teaching logic to support students, developed by Geck et al [11]. Iltis supports the modeling process of propositional logic through a task-based exercise framework, where teachers specify multi-step exercises in XML and students receive immediate and structured feedback. The feedback mechanism is notably sophisticated as the system attempts to identify specific student misconceptions and provides targeted explanations, instead of just marking answers correct or incorrect. This has proven meaningful in practice as studies showed that students who received detailed feedback performed much better, having lower error rates. However, there are several limitations with this application: the feedback mechanism performs poorly when the students makes more than two mistakes simultaneously, and the evaluation methodology was imperfect due to variation in exercise phrasing groups. At the time of publication, the prototype only supported propositional logic, with the modal and first-order logic extensions still being underdeveloped. Iltis is also only grounded in classical monotonic logic, and offers no support for defeasible reasoning [11].

Another relevant pedagogical tool is LogicLearner, developed by Inamder et al. [13], which is web-based application designed to support undergraduate students in practicing propositional logic proofs. This tool provides a step-by-step interactive proof environment, which is backed by an AI proof solver, allowing students to request hints on demand. Empirical evaluation was done across two semesters of a discrete mathematics course at Columbia University, which demonstrated improvements in exam performance, and the tool was found to outperform LLM-based tools such as ChatGPT for guided proof solving. The authors claim that this is the first and only application which offers fully guided proof-solving at the time of writing. However, it has notable limitations, where its AI heuristic does not guarantee accuracy, and more critically, it is designed for classical propositional logic, and not for defeasible reasoning [13]. LogicLearner provides a useful architectural and pedagogical reference point, but does not address the learning needs of students and researchers engaging with defeasible logic.

5.2 Defeasible Reasoners

Beyond pedagogical tools for classical logic, it is also important to examine existing tools that support defeasible reasoning, even though they are primarily designed for research purposes rather than teaching. The Tweety Project, developed by Thimm [22], is an open source Java library built for scientific experimentation with knowledge representation formalisms. It supports over 15 different formalisms including defeasible logic programming, conditional logic and probabilistic reasoning, making it one of the most

comprehensive frameworks of its kind. Tweety follows a formal declarative structure, where users define formulas at the syntactic level, collect them into knowledge bases, and derived new conclusions using a specialised reasoners. The authors describe that its a general-purpose framework, capable of handling a wide variety of knowledge representation formalisms. However, despite its expressive power, the Tweety Project has several limitations from a pedagogical perspective. It is primarily designed for researchers with strong programming expertise, as interaction occurs through Java APIs. At the time of publication, no fully developed graphical or interactive interface was available, with command line and web-based interfaces still under development. The Tweety project provides no pedagogical support, as there are no guided exercises, feedback mechanisms, or instructional scaffolding to support learners attempting to understand defeasible reasoning [22].

While DR-Device, developed by Bassiliades et al [1], offers a more accessible interface than Tweety, it is a research tool instead of a pedagogical one. DR-Device is a web-based defeasible reasoning system that supports rule-based reasoning with incomplete and inconsistent information, with applications in ontology integration and the modeling of business rules. It supports the three core rule types of defeasible logic, which are strict rules, defeasible rules, and defeaters, as well as superiority relations for resolving conflicts between rules [1]. Rules can be expressed in either a native CLIPS-like syntax or a RuleML-based syntax. Despite being more interface-accessible than Tweety, DR-Device is designed for Semantic Web applications rather than education. It provides no guided exercises, feedback mechanisms, and requires some familiarity with formal rule syntax, making it unsuitable for students engaging with defeasible reasoning for the first time [1].

The tools examined in this section reveal a clear and consistent gap. Pedagogical tools such as Iltis and LogicLearner demonstrated that structured, feedback driven environments can meaningfully support students who are learning classical logic, but neither support defeasible reasoning. On the other hand, defeasible reasoning can be implemented in expressive and capable systems, such as the Tweety Project and DR-Device, but neither is designed with learning in mind. No existing tool bridges both sides of this gap. To the best of our knowledge, there is no dedicated pedagogical tool designed specifically for defeasible reasoning, which motivates the development of this tool presented in this work.

6 Conclusion

This literature review has examined the theoretical and practical foundations relevant to the development of a pedagogical tool for defeasible reasoning. Beginning with classical propositional logic, the limitations of monotonic reasoning were identified, motivating the need for nonmonotonic reasoning, and more specifically, defeasible reasoning. The KLM framework was then examined as the formal foundation for defeasible reasoning, providing the semantic grounding for defeasible conditionals and entailment.

A key insight from this review is that defeasible reasoning does not need to be treated as entirely separate from classical reasoning. Through materialisation, defeasible statements can be converted into classical implications, and the Rational Closure algorithm reduces the defeasible entailment checking into a series of classical entailment checks. This connection is pedagogically significant, as it allows students with a background in propositional logic to approach defeasible reasoning through concepts that they already understand, instead of encountering it as an entirely new system.

The review of existing tools revealed a consistent gap in the literature. Pedagogical tools such as Iltis and LogicLearner demonstrate the value of structured, feedback driven learning environments for classical logic, while defeasible reasoners such as DR-Device and Tweety demonstrate that defeasible reasoning can be implemented in expressive systems. However, no existing tool bridges the two types of tools. The absence of a dedicated pedagogical tool for defeasible reasoning represents a clear gap, and one that remains open in the literature.

References

[1] Nick Bassiliades, Grigoris Antoniou, and Ioannis Vlahavas. 2004. DR-DEVICE: A Defeasible Logic System for the Semantic Web. In *Principles and Practice of Semantic Web Reasoning*, Hans Jürgen Ohlbach and Sebastian Schaffert (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 134–148. https://doi.org/10.1007/978-3-540-30122-6_10

[2] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science.* Springer, New York.

[3] Giovanni Casini, Thomas Meyer, Kodylan Moodley, and Riku Nortjé. 2014. Relevant Closure: A New Form of Defeasible Reasoning for Description Logics. In *Logics in Artificial Intelligence*, Eduardo Fermé and João Leite (Eds.). Springer International Publishing, Cham, 92–106. https://doi.org/10.1007/978-3-319-11558-0_7

[4] Giovanni CASINI, Thomas Meyer, and Ivan Varzinczak. 2018. Defeasible Entailment: from Rational Closure to Lexicographic Closure and Beyond. In *Proceeding of the 17th International Workshop on Non-Monotonic Reasoning (NMR 2018)* (Tempe, United States - Arizona). CE - Commission Européenne. hdl:https://orbilu.uni.lu/handle/10993/37393 http://www4.uma.pt/nmr2018/NMR2018Proceedings.pdf

[5] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2019. Taking Defeasible Entailment Beyond Rational Closure. In *Logics in Artificial Intelligence*, Francesco Calimeri, Nicola Leone, and Marco Manna (Eds.). Springer International Publishing, Cham, 182–197.

[6] Giovanni Casini and Umberto Straccia. 2010. Rational Closure for Defeasible Description Logics. In *Logics in Artificial Intelligence*, Tomi Janhunen and Ilkka Niemelä (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 77–90.

[7] Sandra K. Dewitz, Young Ryu, and Ronald M. Lee. 1994. Defeasible reasoning in law. *Decision Support Systems* 11, 2 (1994), 133–155. doi:10.1016/0167-9236(94)90029-9

[8] Kenneth G. Ferguson. 2003. Monotonicity in Practical Reasoning. *Argumentation* 17, 3 (Sept. 2003), 335–346. doi:10.1023/A:1025164703468

[9] Michael Freund. 1998. Preferential reasoning in the perspective of Poole default logic. *Artificial Intelligence* 98, 1 (1998), 209–235. doi:10.1016/S0004-3702(97)00053-2

[10] Jean H Gallier. 2015. *Logic for computer science: foundations of automatic theorem proving.* Courier Dover Publications.

[11] Gaetano Geck, Artur Ljulin, Sebastian Peter, Jonas Schmidt, Fabian Vehlken, and Thomas Zeume. 2018. Introduction to Iltis: an interactive, web-based system for teaching logic. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education* (Larnaca, Cyprus) *(ITiCSE 2018)*. Association for Computing Machinery, New York, NY, USA, 141–146. doi:10.1145/3197091.3197095

[12] Chipo Hamayobe. 2026. *Implementation and Evaluation of KLM-Style Defeasible Entailment and Explanation Algorithms.* Master of Science Dissertation. University of Cape Town, Cape Town, South Africa. Supervisors: Thomas Meyer and Jonas Philipp Haldiman.

[13] Amogh Inamdar, Uzay Macar, Michel Vazirani, Michael Tarnow, Zarina Mustapha, Natalia Dittren, Sam Sadeh, Nakul Verma, and Ansaf Salleb-Aouissi. 2025. LogicLearner: A Tool for the Guided Practice of Propositional Logic Proofs. arXiv:2503.19280 [cs.DM] https://arxiv.org/abs/2503.19280

[14] A. Kaliski. 2020. *An Overview of KLM-Style Defeasible Entailment.* Master's thesis. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/32743

[15] Robert Koons. 2025. Defeasible Reasoning. In *The Stanford Encyclopedia of Philosophy* (Summer 2025 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University.

[16] Sarit Kraus, Daniel Lehmann, and Menachem Magidor. 1990. Nonmonotonic reasoning, preferential models and cumulative logics. *Artificial Intelligence* 44, 1 (1990), 167–207. doi:10.1016/0004-3702(90)90101-5

[17] Daniel Lehmann. 1995. Another Perspective on Default Reasoning. *Annals of Mathematics and Artificial Intelligence* 15, 1 (March 1995), 61–82. doi:10.1007/BF01535841

[18] Daniel Lehmann and Menachem Magidor. 1992. What Does a Conditional Knowledge Base Entail? *Artificial Intelligence* 55, 1 (1992), 1–60. doi:10.1016/0004-3702(92)90041-u

[19] Luca Longo and Pierpaolo Dondio. 2014. Defeasible Reasoning and Argument-Based Systems in Medical Fields: An Informal Overview. In *2014 IEEE 27th International Symposium on Computer-Based Medical Systems*. 376–381. doi:10.1109/CBMS.2014.126

[20] John L. Pollock. 1987. Defeasible reasoning. *Cognitive Science* 11, 4 (1987), 481–518. doi:10.1016/S0364-0213(87)80017-4

[21] Raymond Reiter. 1988. Chapter 12 - Nonmonotonic Reasoning. In *Exploring Artificial Intelligence*, Howard E. Shrobe and the American Association for Artificial Intelligence (Eds.). Morgan Kaufmann, 439–481. doi:10.1016/B978-0-934613-67-5.50016-2

[22] Matthias Thimm. 2017. The Tweety Library Collection for Logical Aspects of Artificial Intelligence and Knowledge Representation. *KI - Künstliche Intelligenz* 31, 1 (March 2017), 93–97. doi:10.1007/s13218-016-0458-4